

OpenCV and CNN in Deep Learning

OpenCV :

- **OpenCV (Open Source Computer Vision Library)** is a powerful library for image processing and computer vision.
- Written in C++ with Python bindings.
- Used in:
 - Image transformations (resize, rotate, filter).
 - Feature detection (edges, corners, faces).
 - Video analysis.
 - Object recognition (integrated with CNN/DL models).

Key Concepts

- **Image Representation:** Stored as a matrix (pixels). Each pixel has values for **BGR** (Blue, Green, Red) in OpenCV.
- **Grayscale Conversion:** Simplifies image → single intensity value (0–255).
- **Edge Detection:** Finds sudden intensity changes (object boundaries).
- **Integration with Deep Learning:** OpenCV provides pre-processing before sending images to CNNs.

CNN (Convolutional Neural Network)

- **CNN** is a type of deep learning model specialized for images.
- Learns features **automatically** (edges → shapes → objects).
- Key layers:
 - **Convolution Layer:** Extracts features using filters.
 - **Pooling Layer:** Reduces size, keeps important features.
 - **Fully Connected Layer:** Classifies objects.
- Used in:
 - Image classification.
 - Object detection.
 - Face recognition.

Code Explanations

OpenCVPixel.py

Concept: Show image pixel values.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read image
img = cv2.imread("sample.jpg")

# 2. Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Display image + pixel grid
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(gray, cmap="gray");
plt.title("Grayscale Image"); plt.axis("off")
plt.subplot(1,2,2); plt.imshow(gray, cmap="gray");
plt.colorbar(label="Pixel Value"); plt.title("Pixel
Values") plt.show()
```

Learning:

- How images are stored as pixel grids.
- Visualization of intensity values.

Edge.py

Concept: Detect edges in an image.

```
import cv2

# Read image
img = cv2.imread("sample.jpg", 0) # grayscale

# Canny Edge Detection
edges = cv2.Canny(img, 100, 200)

cv2.imshow("Original", img)
cv2.imshow("Edges", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Learning:

- Edge detection helps CNN identify **boundaries of objects**.

- OpenCV's **Canny** algorithm works in 3 steps:

1. Noise reduction.
2. Gradient calculation.
3. Detecting edges.

CNNedgeDetection.py

Concept: Using CNN for edge detection/classification.

```
import cv2
import numpy as np
from tensorflow.keras.applications import MobileNetV2,
preprocess_input, decode_predictions

# Load pre-trained CNN
model = MobileNetV2(weights="imagenet")

# Capture from webcam
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img, (224, 224))
    x = np.expand_dims(img_resized, axis=0).astype(np.float32)
    x = preprocess_input(x)

    # Prediction
    preds = model.predict(x, verbose=0)
    decoded = decode_predictions(preds, top=1)[0][0]
    label = f"{decoded[1]}: {decoded[2]*100:.1f}%"

    # Show on screen
    cv2.putText(frame, label, (16, 40), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,255,0), 2)
    cv2.imshow("CNN Classification", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Learning:

- Webcam captures **real-time frames**.
- Frames → CNN (MobileNetV2) → Predict object.
- CNN automatically detects **edges** → **shapes** → **object label**.

